

Apuntes teorico 25/03

materia: metodologia 2

profesor: Arce

TEMAS : _____

Patrones de diseño - web:



**REFACTORING
· GURU ·**

★ **Contenido premium**

🔧 **Patrones de diseño**

- Introducción
- Catálogo
- Patrones creacionales
 - **Factory Method**
 - Abstract Factory
 - Builder
 - Prototype
 - Singleton
- Patrones estructurales
- Patrones de comportamiento
- Ejemplos de código

🔗 **Refactorización**
(próximamente)

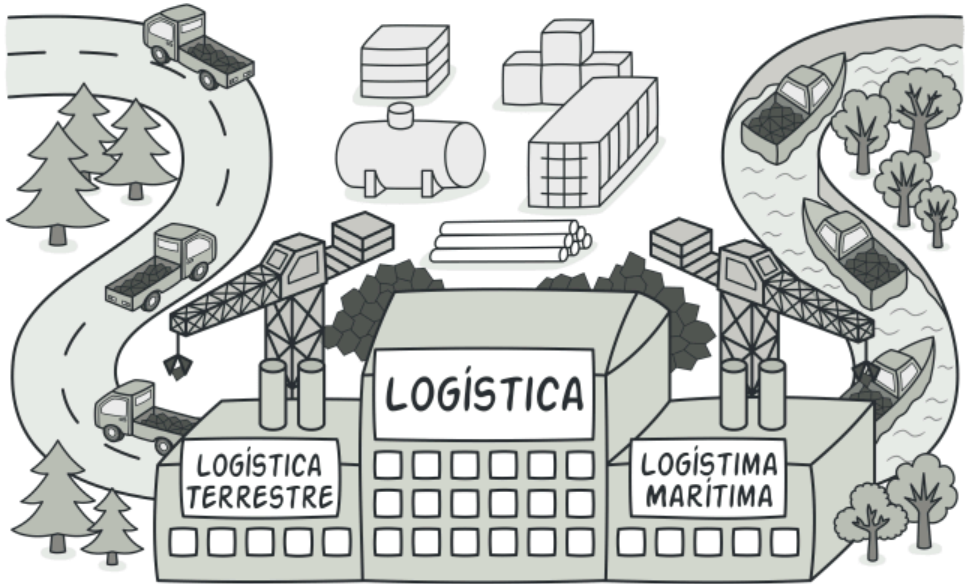
🏠 / [Patrones de diseño](#) / [Patrones creacionales](#)

Factory Method

También llamado: Método fábrica, Constructor virtual

🗨 Propósito

Factory Method es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.



QUE CUERNOS SON LOS PATRONES DE DISEÑO ?!

Los patrones de diseño son como recetas o trucos ya probados para resolver problemas que aparecen todo el tiempo cuando estás programando.

No son código que copias y pegás directamente. Son más como planos o formas inteligentes de organizar el código para que no se vuelva un quilombo.

Ejemplo visual !

si querés hacer una casa, entonces no inventás todo desde cero cada vez. Usás planos que ya saben cómo poner las paredes, las ventanas y la puerta para que no se caiga.

En programación pasa lo mismo: hay problemas que se repiten (crear objetos, conectar pantallas, guardar datos, etc.). Los patrones son las soluciones que miles de programadores ya descubrieron y que funcionan bien.

¿Para qué nos sirven en la programación?

- pa No reinventar la rueda cada vez
- pa Hacer que el código sea más fácil de mantener y cambiar
- pa Hablar el mismo idioma con otros programadores

con esto se ahorra tiempo y es util saberlo porque en entrevistas laborales los preguntan

algunos metodos que mencionó el profe en clase:

Factory Method (Fábrica de objetos)

una cafetería que vende diferentes tipos de café:

- Café Americano
- Café Latte
- Café Cappuccino
- Café Mocha

Cada café se prepara de forma distinta (diferente cantidad de leche, espuma, chocolate, etc.) La Cafeteria es la fábrica. El método `crear_cafe()` es el Factory Method. Cada tipo de café (Americano, Latte, etc.) son los productos concretos. El cliente (la app, el barista o el cajero) solo dice: "quiero un latte" y la fábrica se encarga del resto.

Paso a paso :

1. **Producto común:** Todos los cafés saben hacer las mismas cosas:
 - `preparar()`
 - `servir()`
2. **Fábrica :** Tiene un método llamado `crear_cafe()` que decide qué café crear.

otro ejemplo de aplicación: Una app que genera **documentos** (PDF, Word, Excel) según lo que el usuario elija.

link a la parte de la web que explica el metodo: [🌐 Factory Method](#)

Singleton “Solo una sola copia en toda la app”

ejemplo visual!

Imaginate esto:

En tu casa hay **un solo control remoto** de la tele. Si cada persona de la familia tuviera su propio control remoto, se armaría un lío terrible: uno sube el volumen, otro cambia de canal, otro lo pierde... todo se descontrola.

Por eso decidís: **“Va a haber solo UN control remoto y todos vamos a usarlo”.**

Eso es exactamente lo que hace el **Singleton**: ***Garantiza que solo exista una única instancia de una clase en toda tu aplicación, y que cualquiera pueda acceder a ella fácilmente.***

5 ejemplos reales donde es muy conveniente usar Singleton

1. **Conexión a la base de datos**
 2. **Logger / Registro de errores** : Querés que todos los mensajes de error se escriban en un solo archivo, no que cada módulo cree su propio logger.
 3. **Configuración de la aplicación** : Colores, idioma, rutas a carpetas, claves API... todo en un solo lugar.
 4. **Gestor de impresora** en un sistema de facturación : Solo hay una impresora física.
-

Observer – “Cuando algo cambia, avísame automáticamente”

EJEMPLO VISUAL!

Estás en un grupo de WhatsApp. Cuando uno sube una foto o cambia el estado, todos los que están en el grupo reciben la notificación automáticamente.

No tenés que estar preguntando cada 5 minutos “¿pasó algo nuevo?”. Eso es exactamente el Observer.

Hay dos partes: El Sujeto (el que cambia) : es como el que publica la foto. Los Observadores (los suscriptores) : son los que quieren enterarse cuando pasa algo.

Cuando el sujeto cambia, avisa automáticamente a todos los que se anotaron.

5 ejemplos reales donde es muy útil usar Observer

1. Notificaciones en redes sociales – Cuando alguien te da like o comenta, todos los que siguen esa publicación se enteran.
 2. 2. Juegos multijugador – Cuando un jugador mata al jefe, todos los demás jugadores reciben “¡jefe derrotado!”.
 3. Carrito de compras en una web – Cuando agregás un producto, el contador, el total y la vista del carrito se actualizan solos.
 4. App del clima – Varios widgets y pantallas están suscriptos. Cuando cambia la temperatura, todos se actualizan automáticamente.
 5. Editor de texto o Google Docs – Cuando alguien escribe algo, todos los que están viendo el documento ven el cambio en tiempo real.
-

¿Para qué sirve la **Inyección de Dependencia** en los patrones de diseño?

Ayuda a que los patrones funcionen mejor.

Factory Method

La fábrica crea el objeto, pero gracias a la DI, podés inyectar diferentes fábricas o dependencias a la clase que usa la fábrica. Ejemplo: inyectás la fábrica de cafés según el país (Argentina vs España).

Singleton

El Singleton suele ser una dependencia que se inyecta (por ejemplo la conexión a la BD). En vez de que cada clase llame `DatabaseConnection.getInstance()`, mejor inyectás la misma instancia única desde afuera. Así es más fácil de probar y cambiar.

Observer

El sujeto (el tablero) puede recibir sus observadores por inyección en vez de crearlos adentro. Así podés agregar o quitar observadores (pantalla, app, display) sin tocar el código del tablero.

En resumen: La Inyección de Dependencia hace que tus clases dependan de abstracciones (interfaces) y no de clases concretas. Esto cumple el principio “Inversión de Dependencias” (uno de los SOLID) y hace que todo sea más flexible, fácil de probar y fácil de mantener.

